TRISHEET · DOCUMENT 2 OF 3

How It Works, End to End

What actually happens between typing a ticker and reading a report

---

Follows — one complete journey, from keystroke to finished document

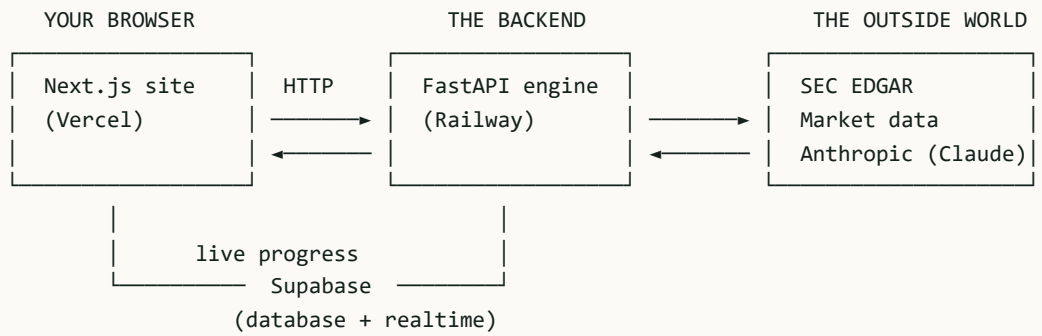
Covers — 13 modules, 8 database tables, 3 output formats

Typical run — 60 to 180 seconds, 40 to 90 sourced facts

Date — 5 August 2026

The shape of the system

> Two programs run in different places. The browser draws screens. The backend does all the work. They never blur — the browser cannot reach the SEC, the language model or the database's write path, and that restriction is what makes the project's guarantees enforceable rather than aspirational.



> **Why the split matters.** If the browser could call the SEC directly, or write facts to the database directly, then every rule about provenance would be a rule the browser could skip. Routing everything through the backend means the write gate is unavoidable.

The journey, in eleven stages

What follows traces one real request: a user types `AAPL` and reads the finished profile.

Stage 1 — Typing

As the user types, the browser asks the backend `GET /resolve/suggest` for matching companies. The backend searches the SEC's official ticker index. A dropdown appears showing company names and tickers.

Files involved: `ticker-combobox.tsx` → `lib/api.ts` → `main.py` → `m01_resolver.py` → `services/edgar.py`

Stage 2 — Confirming which company

On submit, the browser calls `POST /resolve`. The backend turns the input into a confirmed identity:

- **CIK** — the SEC's permanent ID for the company, padded to ten digits.
- **Filer type** — decided by looking at the annual form the company most recently filed. A US filer files a 10-K; a foreign one files a 20-F or 40-F. Everything downstream depends on this, so it is never assumed.
- **Industry code, financial year end, reporting currency** — read from the company's own data, not guessed.

> **If the input matches several companies**, the backend returns the candidates and chooses none. The user picks. Guessing would produce a correct-looking report about the wrong business — the worst possible failure for this product.

Stage 3 — Starting the job

The browser calls `POST /reports` with the CIK and the chosen depth. Here is the key design decision:

> **The request does not wait for the report.** Generating one means reading dozens of documents and writing prose about them — minutes, not seconds. Far longer than a browser will hold a connection open. So the backend registers the job, immediately answers with a job ID, and does the work in the background.

The browser is redirected to `/r/<id>` and starts watching progress. A rate limit caps how many reports one caller may start in a window.

Stage 4 — Extraction: gathering the raw material

The pipeline now runs its modules in a fixed order. Each records what it produced and how long it took.

`m01` Load the company

Confirms the identity again from the authoritative source and loads the full profile. **Required** — if this fails, the report fails.

`m02` Build the filing manifest

Takes the company's entire filing history and narrows it to what this report may use. Drops irrelevant forms. Applies amendment precedence, so a restated figure wins over the original it replaced. Attaches full URLs and accession numbers. **Required.**

`m03` Extract the financial figures

Pulls reported numbers from the SEC's structured XBRL data — revenue, operating income, assets, cash flow, and segment breakdowns. For each metric it tries an ordered list of tags rather than one, because companies tag the same concept differently. It tries the US taxonomy first and falls back to the international one for foreign filers. Duplicates are removed keeping the latest filing. Anything not found becomes a "not disclosed" marker — never an estimate. **Required.**

`m04` Extract the narrative sections

The business description, risk factors and management discussion, as written. No summarising, no ranking, no interpretation. **Optional** — if parsing fails, the report still generates from the figures alone.

`m05` Fetch market data

Share price, market capitalisation, trading multiples. The only module allowed to touch a market provider. Everything it produces is marked Tier 3. **Optional** — if the provider is down, the report generates without the valuation table.

`m13` Read any attached links

If the reader attached a URL, it is fetched and quoted. These become `SourceNote` objects, which by their type *cannot* become facts — so a company web page can never enter the financial highlights table. **Optional.**

Stage 5 — Analysis: all the arithmetic

`m07` Compute every derived figure

Growth rates, margins, ratios, bridges, discounted cash flow, sensitivity and scenario tables — computed in Python with pandas and numpy. Pure functions: same input, byte-identical output, every time. Each derived value carries the formula that produced it so the interface can show its working.

> **This is the single most important rule in the project.** The language model never performs arithmetic. It never recalls, estimates or derives a number. Every figure is computed here, in Python, and handed to the model already calculated. The model's only job is to write sentences about numbers it was given.

`m08` Select comparable companies

Uses a ladder of decreasing authority and records which rung answered: first the compensation peer group named in the proxy statement (a pay committee has stated on the record who it considers peers), then the competition section of the annual report, then industry-code match. **Optional.**

`m09` Build the developments timeline

Dated entries from current-report filings — 8-K for US companies, 6-K for foreign ones — filtered by the filing's own item numbers so only material events appear. **Optional.**

Stage 6 — The write gate

m06 Store the facts — and refuse the bad ones

Everything gathered so far is now offered to the fact store. This is the wall.

Two refusals happen here, each logged rather than silently swallowed:

REFUSAL	WHY
Missing provenance	A fact lacking its tier, source type, source URL, accession number or filing date is discarded. It is never stored blank and never rendered with a gap.
Wrong tier for the section	A Tier 3 or Tier 4 value offered for a financial-highlights metric is hard-blocked. Market data and news can never enter the financial statements section.

The four source tiers

TIER	WHAT IT IS	WHERE IT MAY APPEAR
Tier 1	SEC filings — 10-K, 10-Q, 8-K, DEF 14A, 20-F, 40-F, 6-K	Anywhere, including financial highlights
Tier 2	Company website, investor presentations, press releases	Anywhere, including financial highlights
Tier 3	Market data — price, market cap, multiples	Blocked from financial highlights
Tier 4	News and general web	Blocked from financial highlights

Stage 7 — Writing the prose

m10 The language model writes

The model receives the already-computed facts and writes the narrative around them. What reaches the prompt is fact objects and nothing else — every one has already passed the write gate. No raw web text, no filing HTML and no market payload is ever placed into a prompt here. The model's answer is constrained to a fixed JSON shape so it can be checked rather than trusted.

Stage 8 — Verification: the blocking gate

m11 Verify the prose against the facts

This is the audit, not a control. Findings are graded: blocking findings mark the report as having failed verification and are shown beside the figures they concern; advisory findings are noted without failing the report.

It checks:

- **Every number in the prose exists in the fact store.** A figure matching nothing is a fabrication, whatever produced it.
- **Every figure carries a citation** traceable to the filing it came from.
- **Segment figures sum to the consolidated total** within tolerance.
- **Assets equal liabilities plus equity.**

The results of these checks become the compliance strip the reader sees at the top of the report — fact count, tier distribution, citation coverage, pass or fail.

Stage 9 — Assembly

`m12` Build the outputs

The verified report is turned into three forms: the JSON document the browser renders, a PDF styled as an analyst tearsheet, and an Excel workbook whose ratios are live formulas over the reported figures rather than pasted values. All three are rendered from one canonical document, so a figure cannot appear in the PDF having failed verification for the screen.

Stage 10 — Watching it happen

While all of the above runs, the browser is not idle. Each step writes a record as it starts and settles, and those records stream to the browser live through Supabase Realtime.

The reader sees each step appear with what it produced — "extracted 47 figures", "selected 6 peers", "verification passed". If the live connection is unavailable, the browser falls back to polling the API instead, so the progress screen works either way.

What happens when something fails

The failure policy is stated once, in the pipeline, and applies everywhere:

IF THIS FAILS…	…THEN
SEC EDGAR (resolution, discovery, XBRL)	The report fails, and says why in plain language. Without filings there is nothing to report.
Narrative parsing	The report generates from the figures alone.
Market data	The report generates without the valuation table.
Peer selection	The report generates without the comparison.
PDF or Excel upload	A complete report with a missing download link, and the interface says so.

In every optional case the step settles as skipped or failed with a reason the reader can act on. Nothing renders a blank screen or a stack trace.

Stage 11 — Reading the report

When the run completes, the page renders the finished document.

The provenance rail — the product's identity

Down the right side of the report sits a persistent column of source cards. Each shows the form type, the accession number, the filing date and a link to the document on the SEC's site.

Every figure in the report carries a small superscript number. That number names a card in the rail. Hovering a figure highlights its source. The rail is never collapsed into a footer — it stays visible, because the sourcing *is* the product.

Asking the assistant a question

The reader can ask questions about the report. The assistant answers only from facts that already carry provenance, working in tiers and stopping at the first that has an answer:

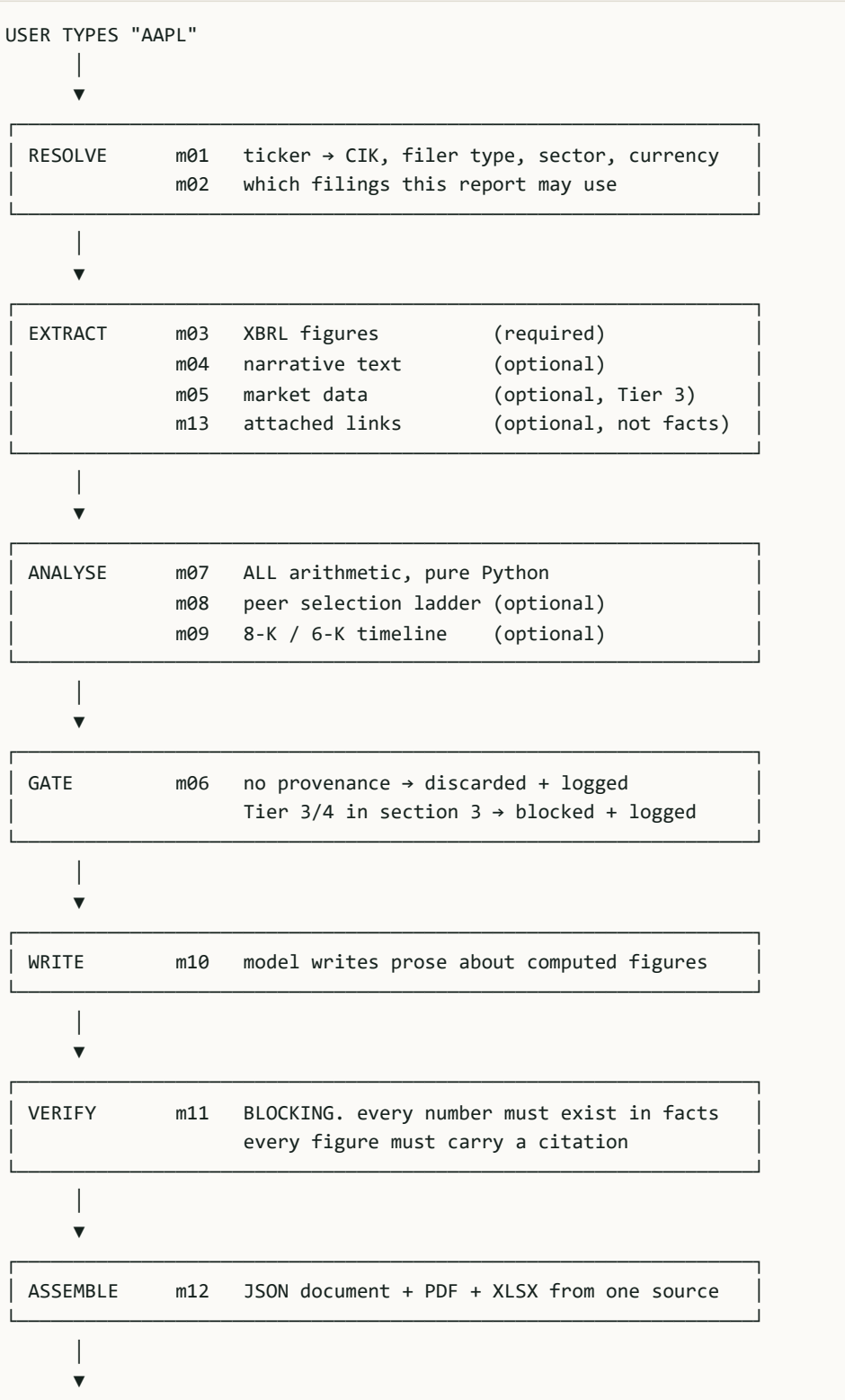
1. **Facts already stored for this report.** Matching is done by plain Python word overlap between the question and each fact's label — a search problem, deliberately not something the language model is asked to decide.
2. **Metrics derivable from those facts**, computed in Python.

Questions asking whether to buy or sell are redirected rather than answered. The system does not give investment advice.

Downloads

The PDF and the Excel workbook are offered from the same verified document. Source references travel with both — the provenance rail is not a screen-only feature.

The whole flow on one page



Read the order again and notice what it guarantees. The model writes at step m10. The gate closes at m06, before it. The audit runs at m11, after it. The model is enclosed on both sides — it cannot introduce a figure, and anything it wrote that does not trace to a stored fact stops the report from rendering at all.